Theory of Automata
CS-3005
Lecture 4

Mahzaib Younas

Lecturer Department of Computer Science

FAST NUCES CFD

Finite Automata

- Finite State Automata (FSA) or Finite State Machine (FSM)
- An FA is a model of a system with discrete inputs and outputs
- The system can be in any ONE of a finite number of the internal configurations or States.
- The states of the system summarizes the information concerning the past inputs that is needed to determine the behavior of the system on subsequent inputs.
- The system is changing the state as a result of new inputs.

Examples

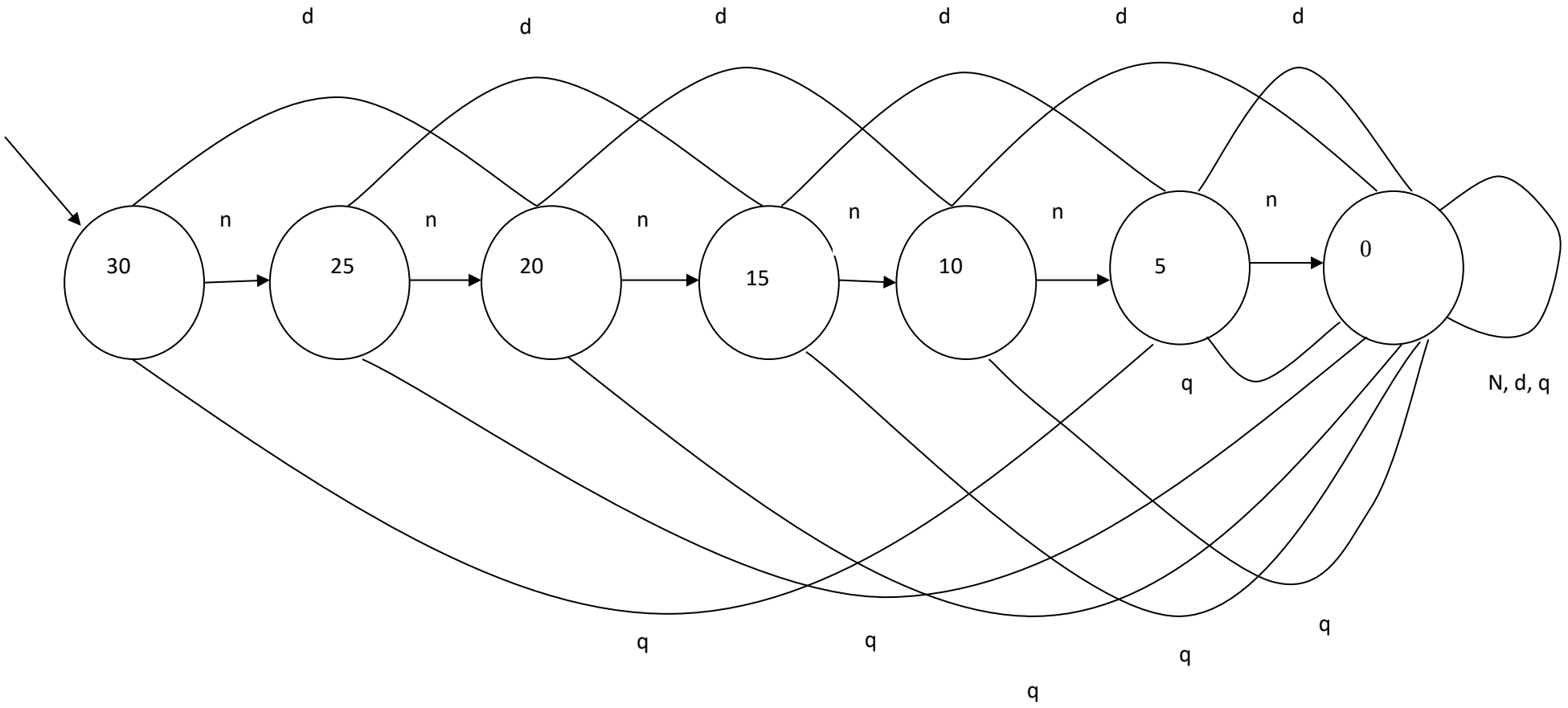
- ON/OFF switch
- Control mechanism of an Elevator
  - It has finite set of states, the floors, to move to
  - It has finite set of inputs,
    - the call buttons on each floor and
    - the floor buttons in the carriage
  - The system only knows the current state/floor it is on
  - After receiving an input the control determines the next state/floor to move to and the direction of the movement in relation to the current floor.

A News Paper wending Machine

- Input to machine consists of *Nickels(5), Dimes(10) and Quarters(25)*
- When 30 cents are inserted, the cover may be opened and a news paper removed
- If total of coins exceed 30 cents, *the machine accepts the over payment and does not give change.*
- Machine has no additional memory, however it knows that an additional 5 cents will unlatch the cover when 25 cents has previously been inserted.

- What is the language of the machine?
- **All strings of *n*, *d*, *q* representing sum of 30 cents or more**
- What are possible states?
- **Need 30 cents, needs 25 cents, ----, needs 5 cents, needs 0 cent, to open the latch**
- What is the final state?
- **need 0 cents**

News Paper Vending Machine FA Model



Model Detail

- Σ = {n, d , q}
- Q = { 0, 5,10, 15, 20, 25, 30)
- F = { 0 }
- S = { 30 }

- Σ = Input Symbols
- Q = Set of states
- F = Final State
- S = Start Space

δ	n	d	q
0	0	0	0
5	0	0	0
10	5	0	0
15	10	5	0
20	15	10	0
25	20	15	0
30	25	20	5

Finite Automata

A **finite automaton** is a collection of three things:

1. A finite set of states, **one** of which is designated as the initial state, called the **start state**, and **some** (maybe **none**) of which are designated as **final states**.

2. An **alphabet Σ** of possible input letters.

3. A finite set of **transitions** that tell for each state and for each letter of the input alphabet which state to go next.

Example 1 :

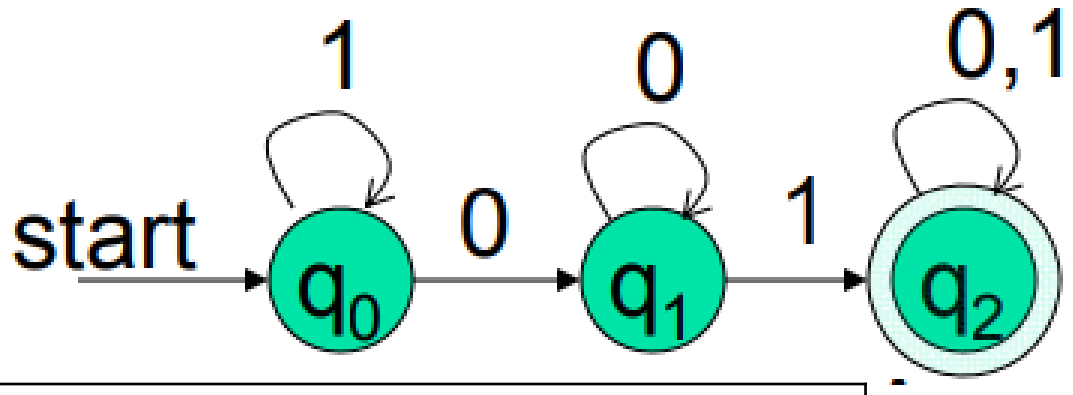
- Build a DFA for the following language:

  L = {w | w is a binary string that contains 01 as a substring}

- Steps for building a DFA to recognize L: Steps for building a DFA to recognize L:
  - ∑ = {0,1}
  - Decide on the states: Q
  - Designate start state and final state(s)
  - δ: Decide on the transitions:
- "Final" states == same as "accepting states"
- Other states == same as "non-accepting states"

DFS for string Containing 01

- Regular expression: (0+1)*01(0+1)*
- Q = $\{q_0, q_1, q_2\}$
- $\sum\{0,1\}$
- Strat Space = q_0
- Final State = q_2



Transition Table		
δ	0	1
q_0	q_1	q_0
q_1	q_1	q_2
q_2	q_2	q_2

Example 2:

Consider the following FA:

- The input alphabet has only the two letters a and b. (We usually use this alphabet throughout the chapter.)

- There are only three states, *x*, *y* and *z*, where *x* is the start state and *z* is the final state.

- The transition list for this FA is as follows:
  - Rule 1: From state *x* and input *a*, go to state y.
  - Rule 2: From state *x* and input *b*, go to state *z*.
  - Rule 3: From state *y* and input *a*, go to state *x*.
  - Rule 4: From state *y* and input *b*, go to state *z*.
  - Rule 5: From state *z* and any input, stay at state *z*.

Example 2:

- Let us examine what happens when the input string '*aaa*' is presented to this FA.
- First input a: state $x \rightarrow y$ by Rule 1.
- Second input *a*: state $y \rightarrow x$ by Rule 3.
- Third input *a*: state $x \rightarrow y$ by Rule 1.
- We did not finish up in the final state z, and therefore have an unsuccessful termination.

Abstract Definition of FA

1. A finite set of states Q = $\{q_0, q_1, q_2\ q_3\ \ldots\}$ of which q_0 is start state.

2. A subset of Q called final state (s).

3. An alphabet $\sum = \{\ x_1, x_2, x_3, \ldots\}$.

4. A transition function δ associating each pair of state and letter with a state:

$$\delta(q,x_j) = x_k$$

Transition Table

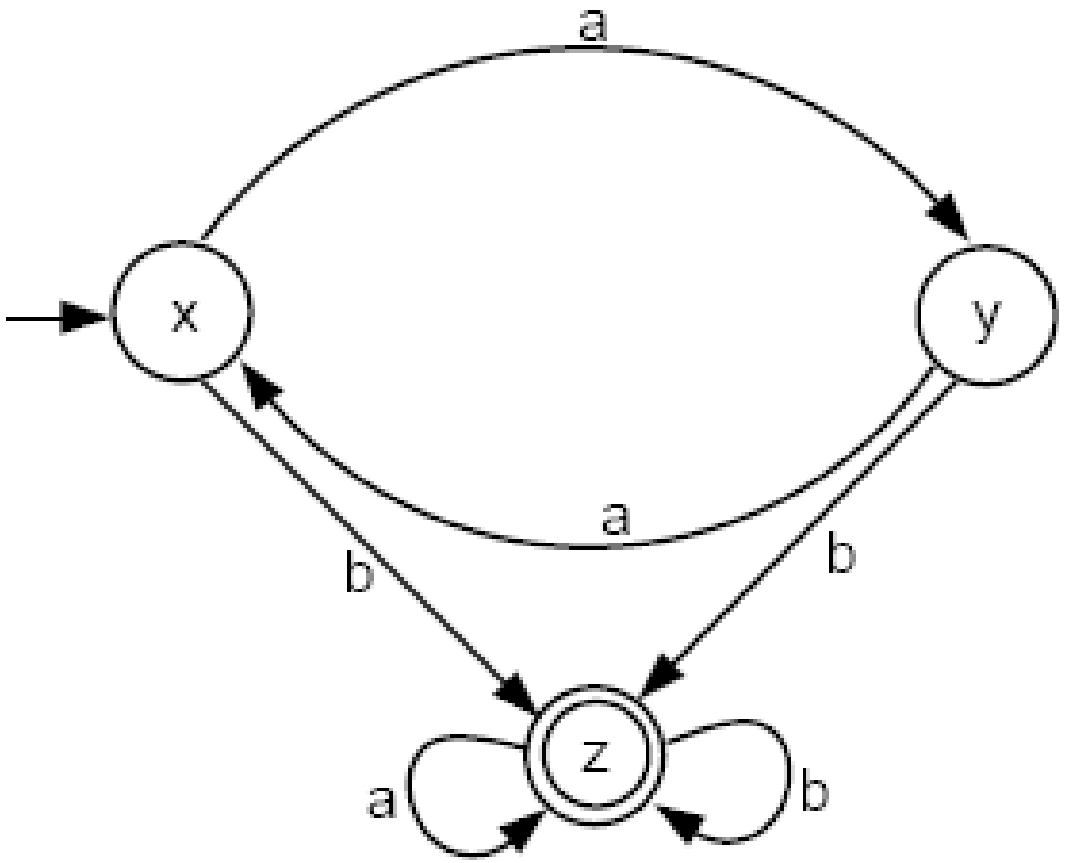
- The transition list can be summarized in a table format in which each row is the name of one of the states, and each column is a letter of the input alphabet.
- For example, the **transition table** for the FA above is

	a	b
Start x	y	z
y	x	z
Final z	z	z

Transition Diagram

- Pictorial representation of an FA gives us more of a feeling for the motion.
- We represent each state by a small **circle**.
- We draw **arrows** showing to which other states the different **input letters** will lead us. We label these arrows with the corresponding input letters.
- If a certain letter makes a state go back to itself, we indicate this by a **loop**.
- We indicate the start state by a **minus sign**, or by labeling it with the word **start**.
- We indicate the final states by **plus signs**, or by labeling them with the word **final**.
- Sometimes, a start state is indicated by **an arrow**, and a final state is indicated by drawing **concentric circles**.

Transition Diagram

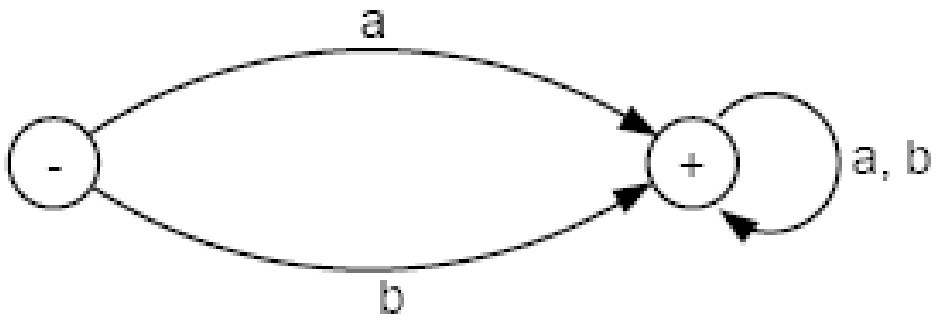


Transition Diagram

- When we depict an FA as circles and arrows, we say that we have drawn a **directed graph**.
- We borrow from Graph Theory the name **directed edge**, or simply **edge**, for the arrow between states.
- ***Every state has as many outgoing edges as there are letters in the alphabet.***
- **It is possible for a state to have no incoming edges or to have many.**

Example – Null String as an input

- By convention, we say that the null string starts in the start state and ends also in the start state for all FAs.
- Consider this FA:



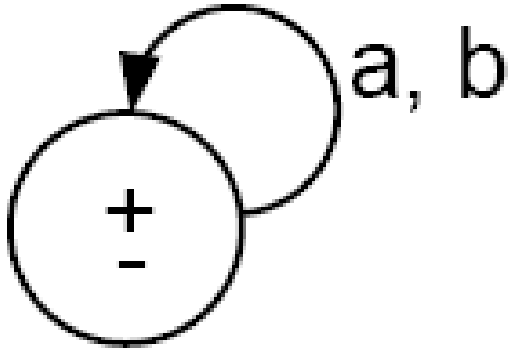
- The language accepted by this FA is the set of all strings except **Λ**.

  The regular expression of this language is

  $(a + b)(a + b)^* = (a + b)^+$

Example

- One of many FAs that accept all words is



- Here, the ± means that the same state is both a start and a final state.
- The language for this machine is

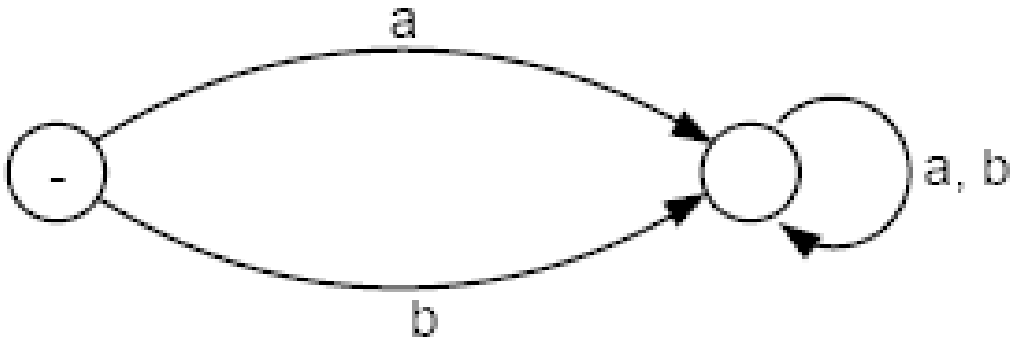
  (a + b)*

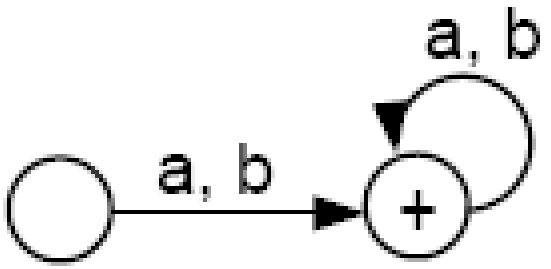
Types of FA

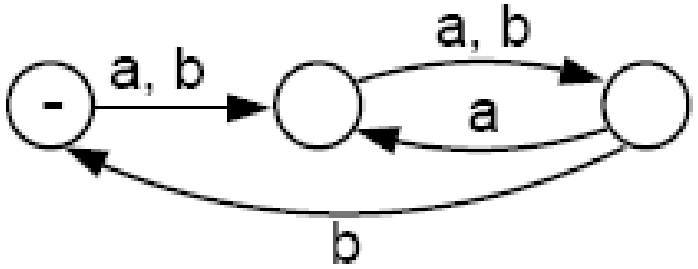
There are FAs that accept no language. These are of two types:

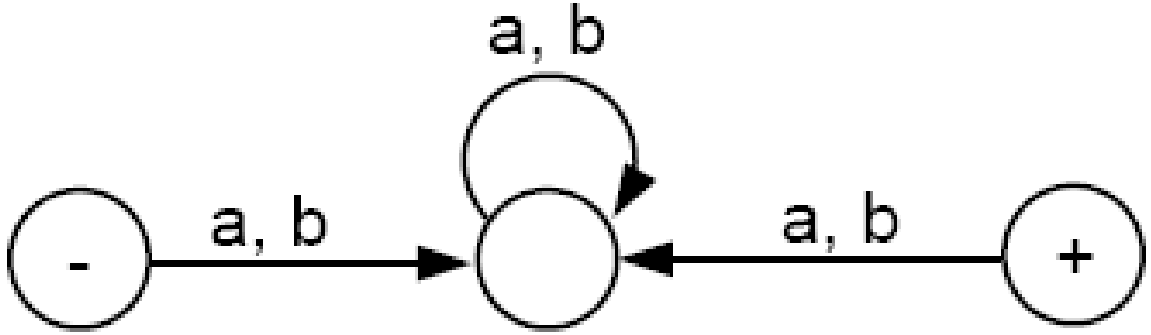
1. The first type includes FAs that have no final states
2. The second type include FAs of which the final states can not be reached from the start state
    1. This may be either because the diagram is in two separate components. In this case, we say that the graph is **disconnected**,
    2. Or it is because the final state has no incoming edges.

Example









FA and their languages

- We will study FA from two different angles:

1. Given a language, can we build a machine for it?

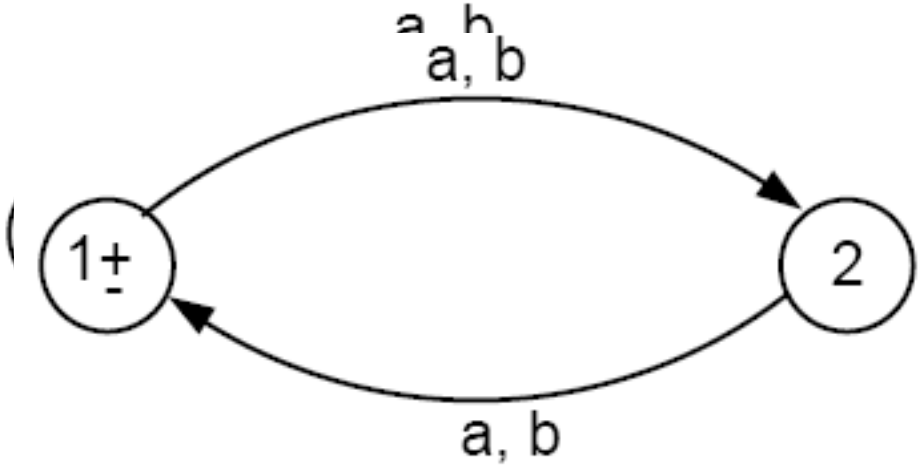
2. Given a machine, can we deduce its language?

Example

- Let us build a machine that accepts the language of all words over the alphabet Σ = {a, b} with an **even number of letters**.
- A mathematician could approach this problem by counting the total number of letters from left to right.
- A computer scientist would solve the problem differently since it is not necessary to do all the counting:
- Use a Boolean flag, named E, initialized with the value TRUE. Every time we read a letter, we reverse the value of E until we have exhausted the input string. We then check the value of E. If it is TRUE, then the input string is in the language; if FALSE, it is not.
- The FA for this language should require only 2 states:
  - State 1: E is TRUE. This is the start and also final state.
  - State 2: E is FALSE.

Example

- So the FA is pictured as follows:



Example 2:

- Let us build a FA that accepts all the words in the language

  a(a + b)*

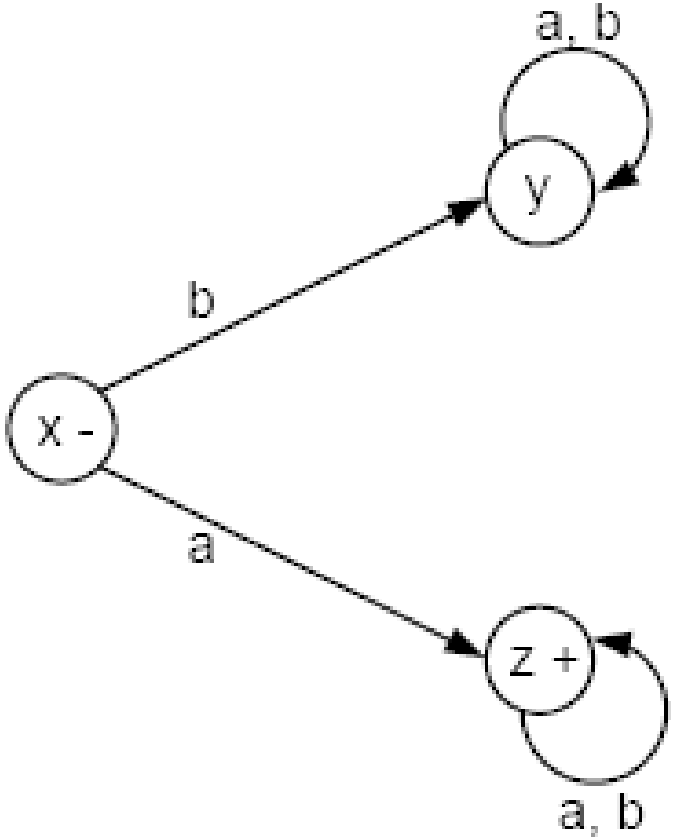
- This is the language of all strings that begin with the letter a.

- Starting at state x, if we read a letter b, we go to a **dead-end** state y. *A dead-end state is one that no string can leave once it has entered.*

- If the first letter we read is an a, then we go to the state z, which is also a final state.

Solution

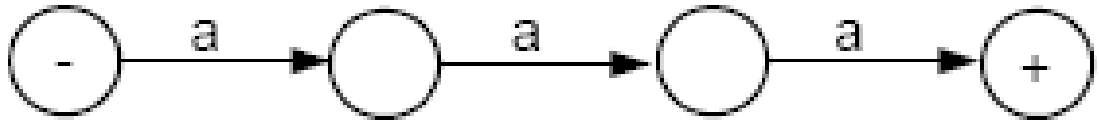
- The example look like



Example 2:

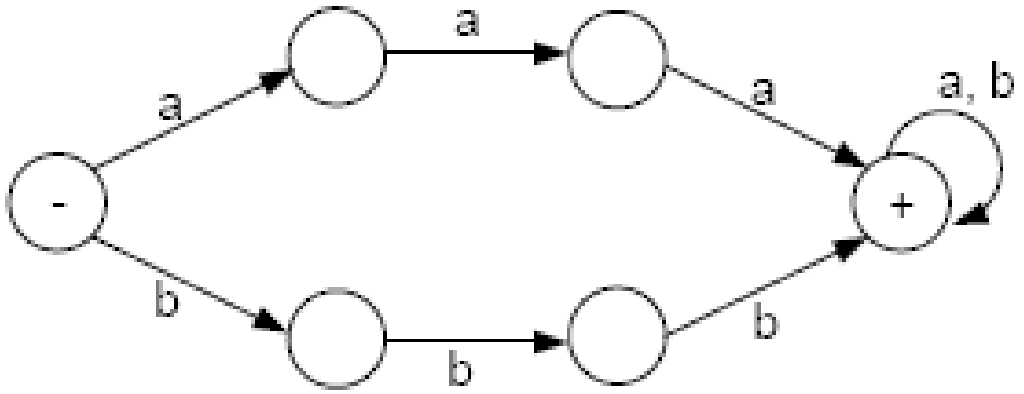
- Let's build a machine that accepts all words **containing a triple letter**, either *aaa* or *bbb*, and only those words.

- From the start state, the FA must have a path of three edges, with no loop, to accept the word *aaa*. So, we begin our FA with the following:



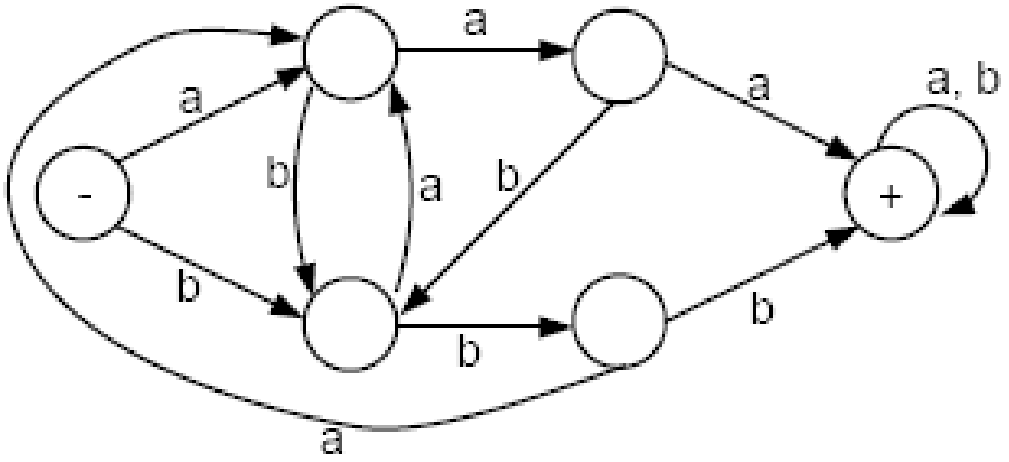
Example 2:

- For similar reason, there must be a path for bbb, that has no loop, and uses entirely differently states. If the b-path shares any states with the a-path, we could mix a's and b's to get to the final state. However, the final state can be shared.



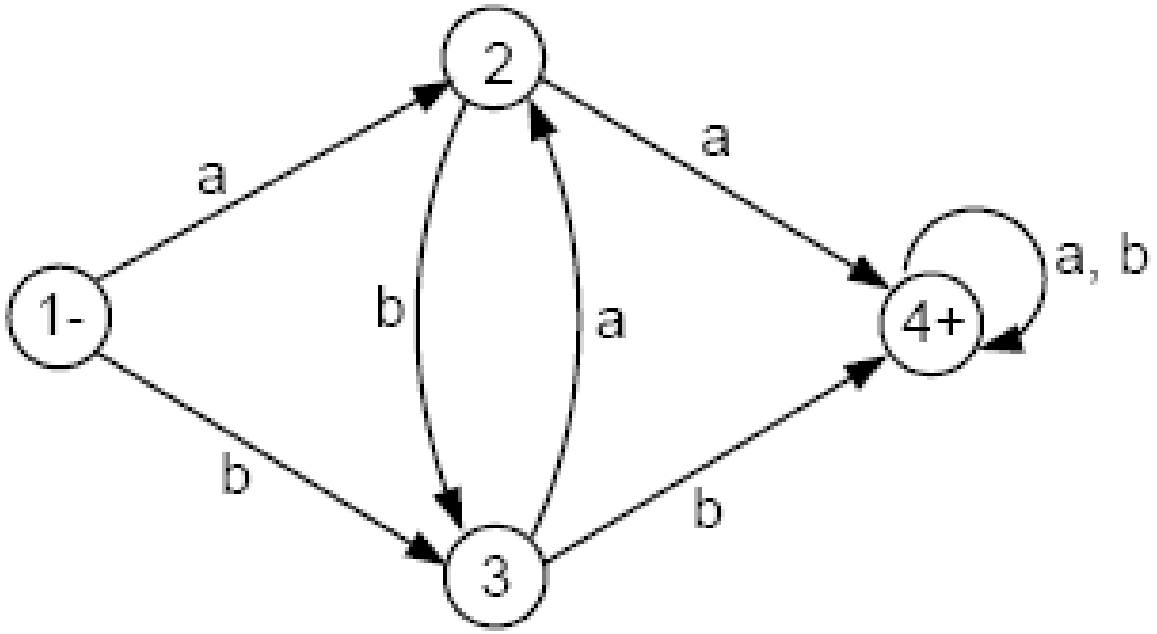
Example 2:

- If we are moving along the a-path and we read a *b* before the third a, we need to jump to the *b*-path in progress and vice versa. The final FA then looks like this:



Example 3:

- Consider the FA below. We would like to examine what language this machine accepts.



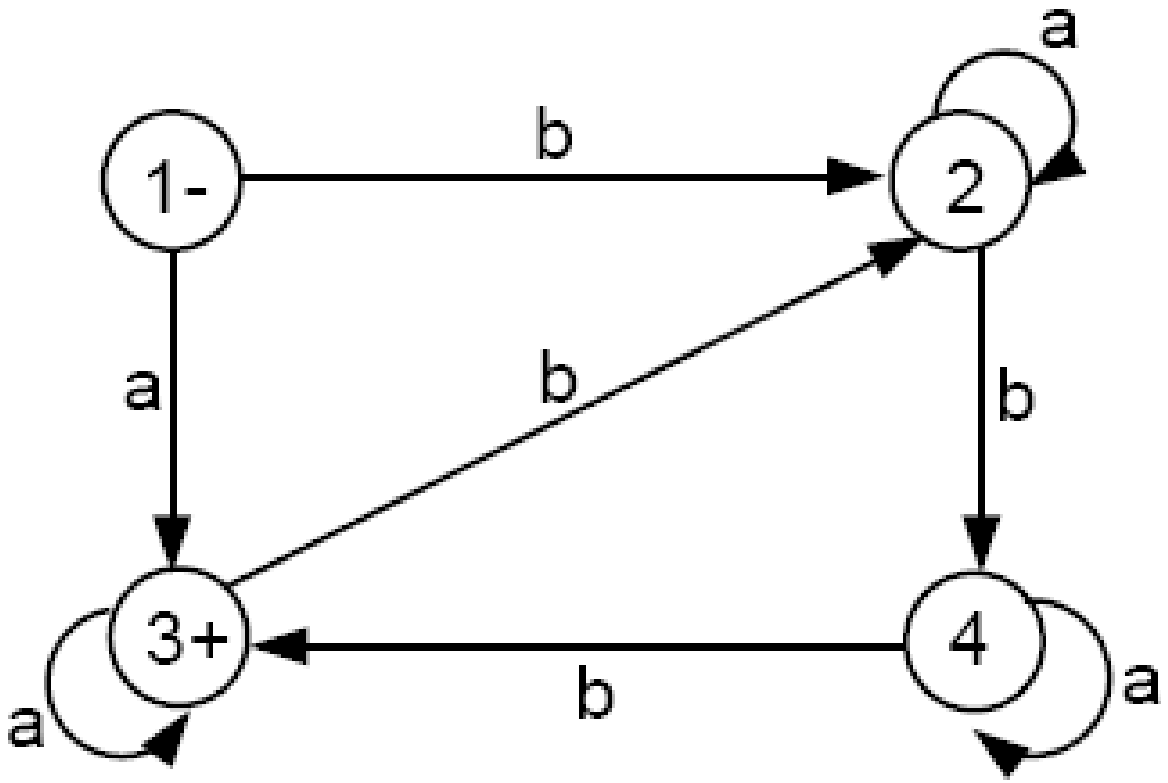
Example:

- There are only two ways to get to the final state 4 in this FA: One is from state 2 and the other is from state 3.
- The only way to get to state 2 is by reading an **a** while in either state 1 or state 3. If we read another **a** we will go to the final state 4.
- Similarly, to get to state 3, we need to read the input letter **b** while in either state 1 or state 2. Once in state 3, if we read another **b**, we will go to the final state 4.
- Thus, the words accepted by this machine are exactly those strings that have a double letter *aa* or *bb* in them. This language is defined by the regular expression

$$(a + b)^*(aa + bb)(a + b)^*$$

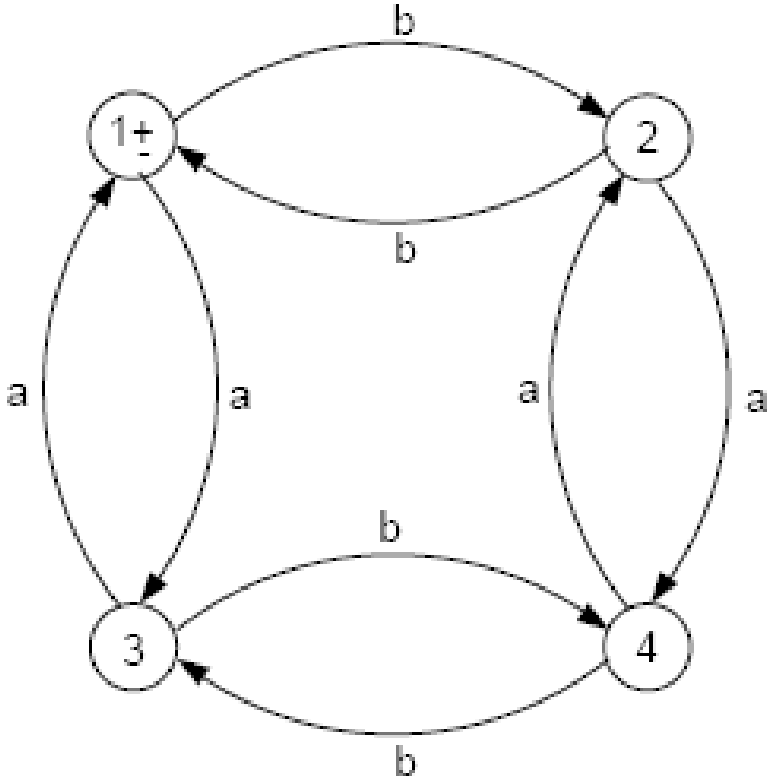
Example 3:

- Consider the FA below. What is the language accepted by this machine?



Example EVEN-EVEN revisited

- Consider the FA below



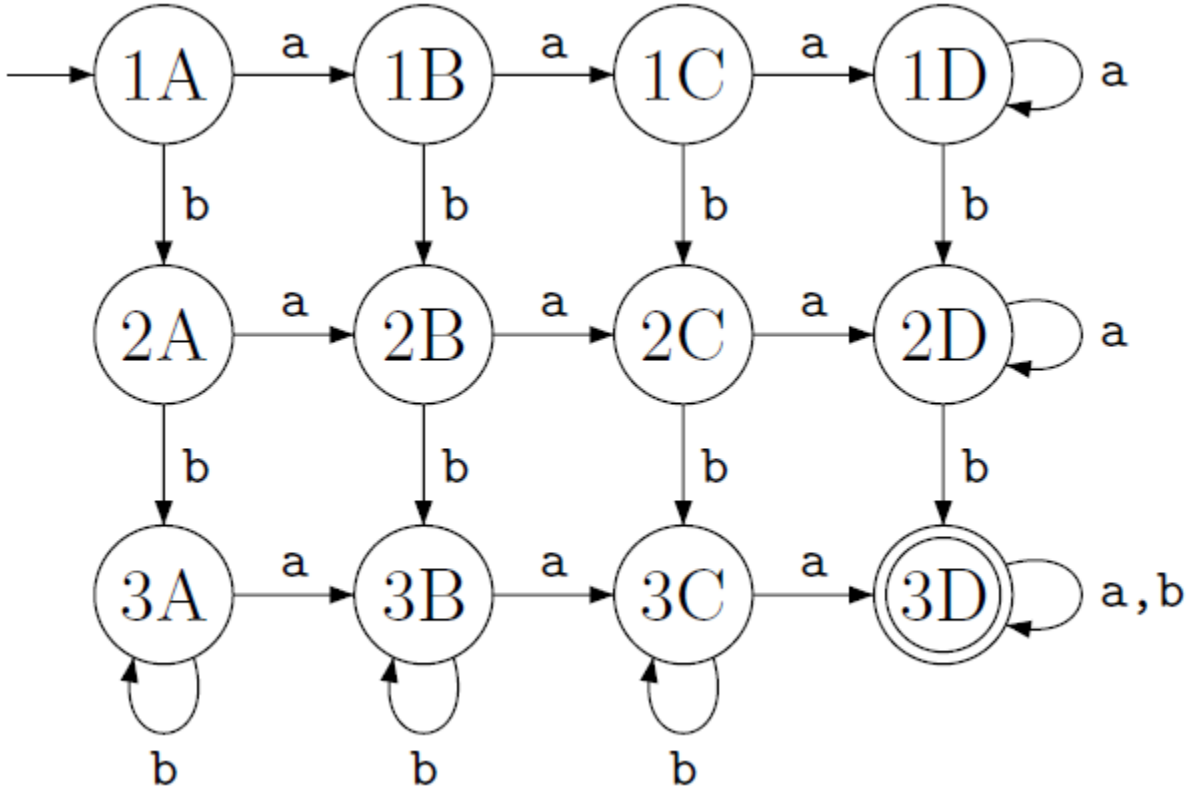
Example EVEN-EVEN revisited

- Therefore, all the words in the language accepted by this FA must have an even number of a's and an even number of b's. So, they are in the language *EVEN-EVEN*.

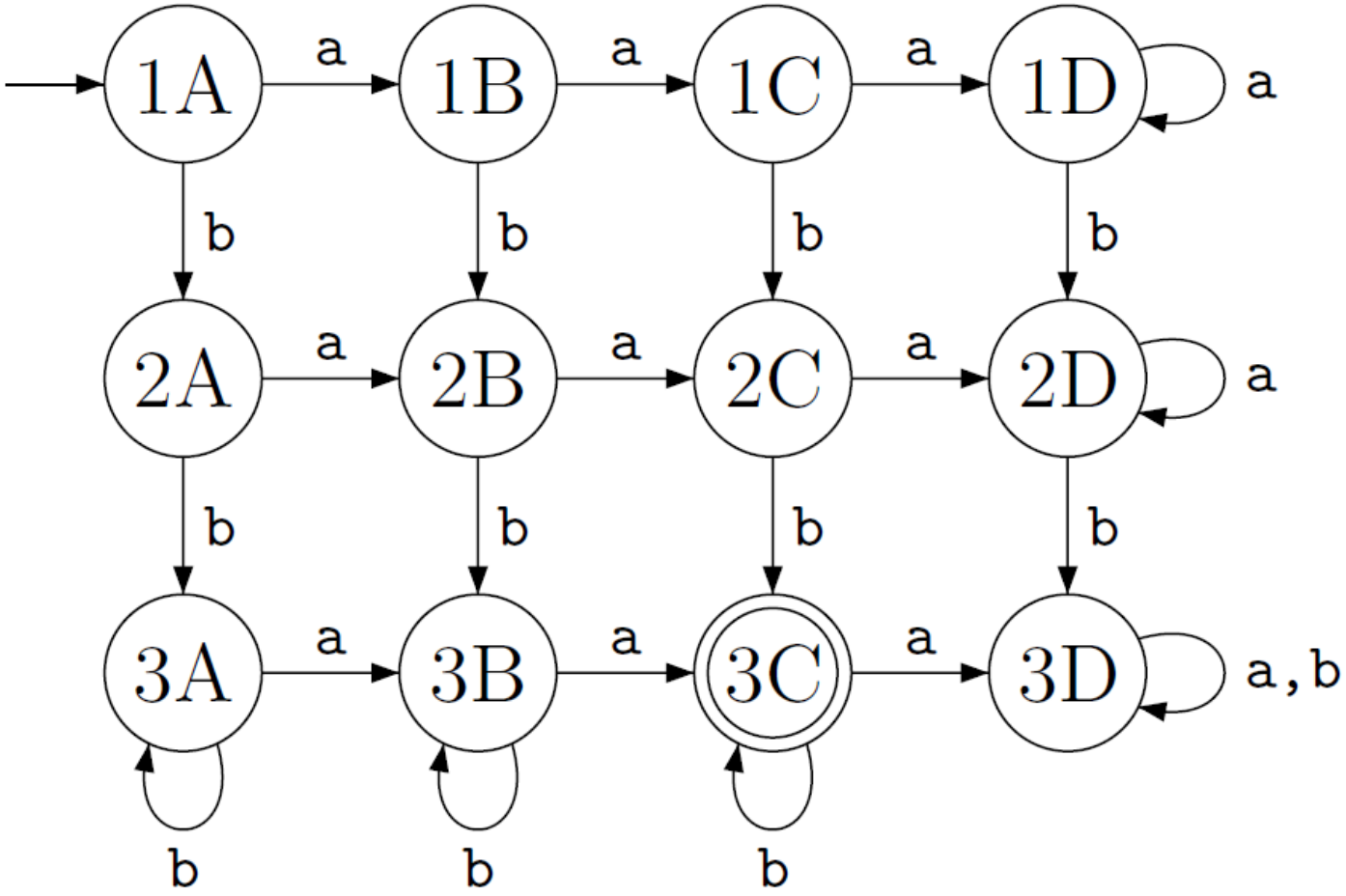
- Notice that all input strings that end in state 2 have an even number of a's but an odd number of b's. All strings that end in state 3 have an even number of b's but an odd number of a's. All strings that end in state 4 have an odd number of a's and an odd number of b's. Thus, every word in the language *EVEN - EVEN* must end in state 1 and therefore be accepted.

- Hence, the language accepted by this FA is *EVEN-EVEN*.

FA – at least three a's and at least two b's



FA – exactly two a's and at least two b's



Some Practice:

- Build an FA that accepts only the language of all words with b as the second letter. Show both the picture and the transition table for this machine and find a regular expression for the language.

Some Practice:

- Build an FA that accepts only the language of all words with b as the second letter. Show both the picture and the transition table for this machine and find a regular expression for the language.
- Regular Expression should be

$$(a+b)\ b\ (a+b)^*$$